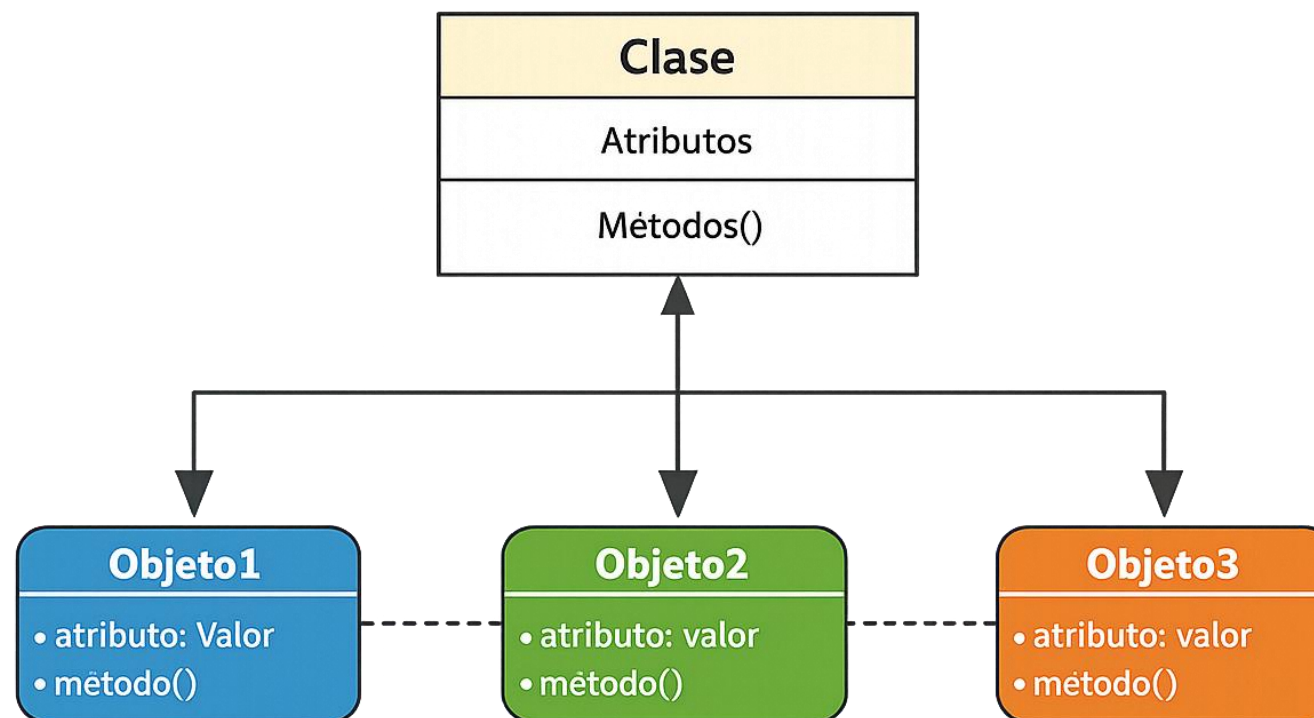




UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

Fundamentos de la Programación Orientada a Objetos

Tema:
TÉCNICAS DE PROGRAMACIÓN.





RESULTADO DE APRENDIZAJE

Interpreta los conceptos fundamentales de la programación orientada a objetos POO para el desarrollo de soluciones computacionales de complejidad media.

Al finalizar la sesión, el estudiante será capaz de:

1

Reconocer las principales técnicas de la Programación Orientada a Objetos.

2

Diferenciar abstracción, encapsulación, herencia y polimorfismo.

3

Relacionar estas técnicas con el desarrollo de aplicaciones en Python.

INTRODUCCIÓN

¿Por qué existen las técnicas de la POO?

A medida que los programas aumentan en tamaño y complejidad, resulta más difícil organizar, modificar y mantener el código.

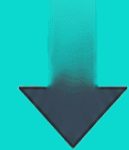


La Programación Orientada a Objetos surge como una forma de estructurar el software mediante objetos que representan elementos del mundo real.

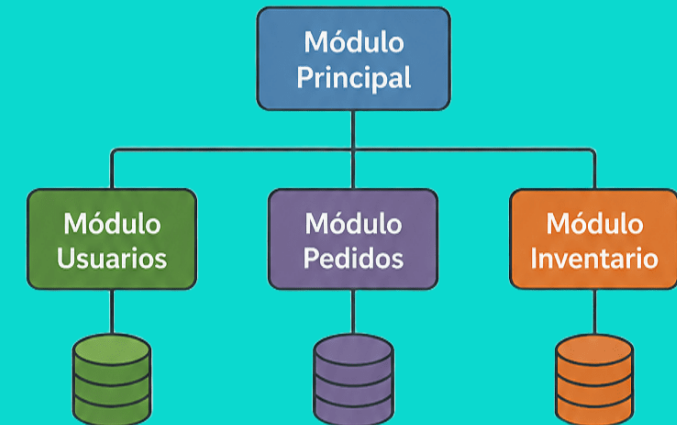
Gracias a ello es posible desarrollar aplicaciones más:

- ✓ **Organizadas**
- ✓ **Reutilizables**
- ✓ **Escalables**
- ✓ **Fáciles de mantener**

Un sistema caótico



Un sistema organizado



LAS BASES DE LA PROGRAMACIÓN ORIENTADA A OBJETOS



¿Qué es una clase?

Una clase es un **molde o plantilla** que define las características (**atributos**) y los comportamientos (**métodos**) que tendrán los objetos creados a partir de ella.

```

class Coche:
    def __init__(self, marca, modelo, año):
        self.marca = marca
        self.modelo = modelo
        self.año = año
        self.encendido = False

    def encender(self):
        self.encendido = True
  
```

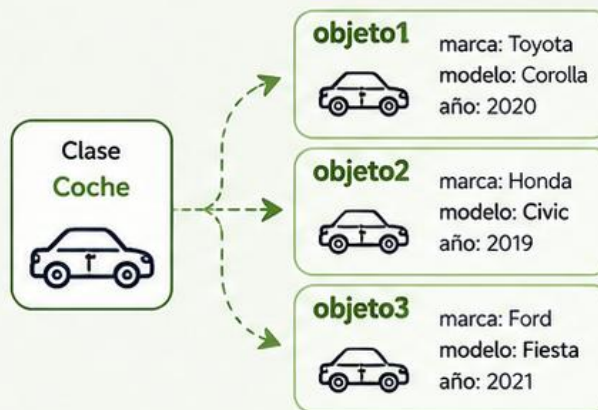


La clase **define qué datos** tiene el objeto y **qué puede hacer**.



¿Qué es un objeto?

Un objeto es una **instancia** de una clase. Es una entidad concreta que tiene valores en sus atributos y puede ejecutar los métodos definidos en la clase.



Cada objeto es **único**, con sus propios valores y estado.



¿Por qué surge la POO?

La POO surge para dar **solución** a los problemas de la programación tradicional cuando los sistemas **crecen en tamaño y complejidad**.



Organización

Permite modelar el mundo real de forma natural y estructurada.



Reutilización

El código se puede reutilizar en diferentes partes del programa o en otros proyectos.



Mantenimiento

Facilita la modificación y expansión del código sin afectar otras partes.



Escalabilidad

Permite construir sistemas más grandes y complejos de manera eficiente.



La POO hace que el código sea más **limpio, modular y fácil de mantener**.



CLASE

Define la estructura y el comportamiento.



OBJETO

Es una instancia concreta con valores reales.



POO EN PYTHON

Nos ayuda a construir programas más eficientes y escalables.

TÉCNICAS FUNDAMENTALES DE LA POO

Video explicativo

ABSTRACCIÓN

Representar lo esencial

Nos enfocamos en las características importantes de un objeto y ocultamos los detalles innecesarios.



Ejemplo en Python

```

class Auto:
    def __init__(self, marca):
        self.marca = marca

    def encender(self):
        print(f"{self.marca} encendido")

# Usamos solo lo esencial
mi_auto = Auto("Toyota")
mi_auto.encender()

> Toyota encendido
  
```

ENCAPSULACIÓN

Proteger los datos

Restringimos el acceso directo a los datos del objeto y los protegemos dentro de la clase.



Ejemplo en Python

```

class CuentaBancaria:
    def __init__(self, saldo):
        self.__saldo = saldo # Atributo privado

    def depositar(self, monto):
        if monto > 0:
            self.__saldo += monto

    def obtener_saldo(self):
        return self.__saldo

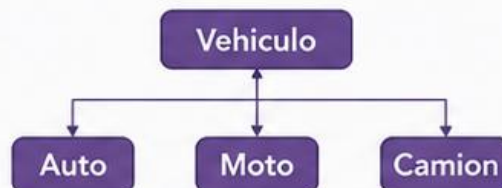
cuenta = CuentaBancaria(1000)
cuenta.depositar(500)
print(cuenta.obtener_saldo())

> 1500
  
```

HERENCIA

Reutilizar código

Creamos nuevas clases basadas en clases existentes para reutilizar atributos y métodos.



Ejemplo en Python

```

class Vehiculo:
    def __init__(self, marca):
        self.marca = marca

    def info(self):
        print(f"Marca: {self.marca}")

class Auto(Vehiculo):
    def __init__(self, marca, puertas):
        super().__init__(marca)
        self.puertas = puertas

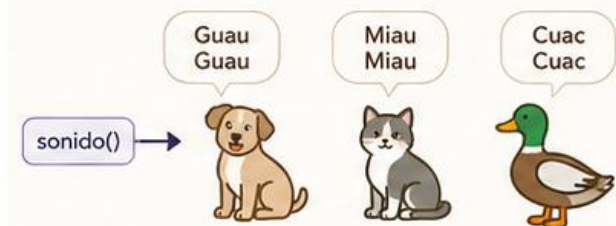
mi_auto = Auto("Honda", 4)
mi_auto.info()

> Marca: Honda
  
```

POLIMORFISMO

Adaptar comportamientos

Distintas clases pueden responder de diferente manera a la misma acción o método.



Ejemplo en Python

```

class Animal:
    def sonido(self):
        pass

class Perro(Animal):
    def sonido(self):
        return "Guau Guau"

class Gato(Animal):
    def sonido(self):
        return "Miau Miau"

for animal in [Perro(), Gato()]:
    print(animal.sonido())

> Guau Guau
> Miau Miau
  
```

ABSTRACCIÓN: QUEDARSE CON LO IMPORTANTE

La abstracción consiste en identificar las características y comportamientos más importantes de un objeto, ignorando los detalles que no son necesarios para resolver un problema.

Ejemplo: 🚗 **Automóvil**

Importa:

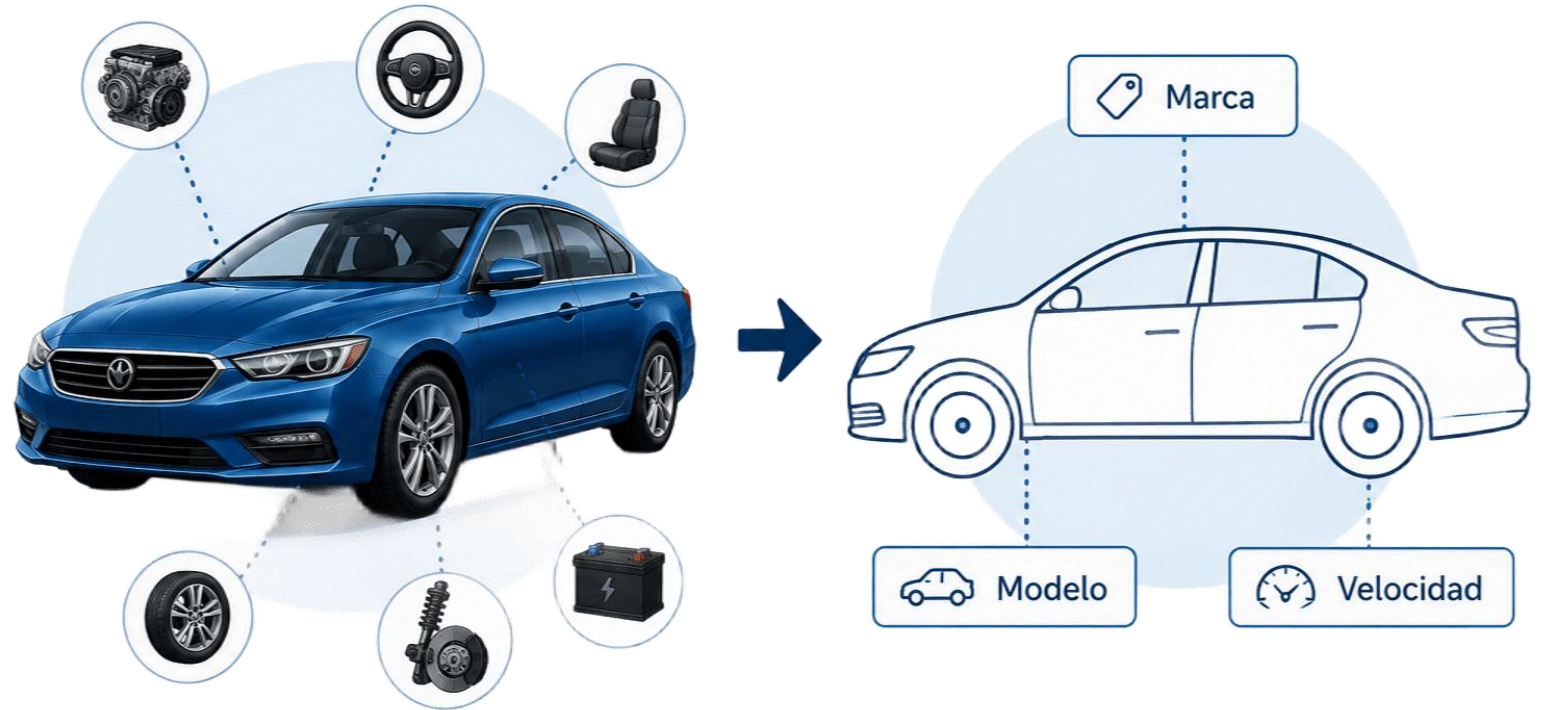
- Marca
- Modelo
- Velocidad

No importa:

- Cada tornillo interno
- El color de los cables del motor

Idea clave:

Nos enfocamos en lo que necesitamos conocer del objeto, no en todos sus detalles.



ENCAPSULACIÓN: PROTEGER LA INFORMACIÓN

La encapsulación consiste en proteger los datos de un objeto y permitir su acceso únicamente mediante acciones controladas.

Ejemplo: 🏦 Cuenta bancaria

Permitido:

- Depositar dinero
- Retirar dinero

No permitido:

- Modificar directamente el saldo

Idea clave:

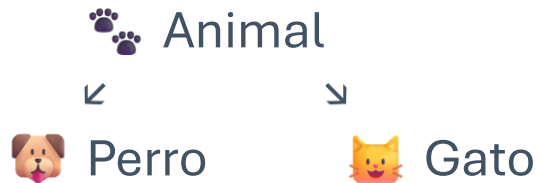
Los datos se protegen para evitar cambios incorrectos o no autorizados.



HERENCIA: REUTILIZAR CARACTERÍSTICAS

La herencia permite crear nuevas clases a partir de otras existentes, reutilizando atributos y comportamientos comunes.

Ejemplo:

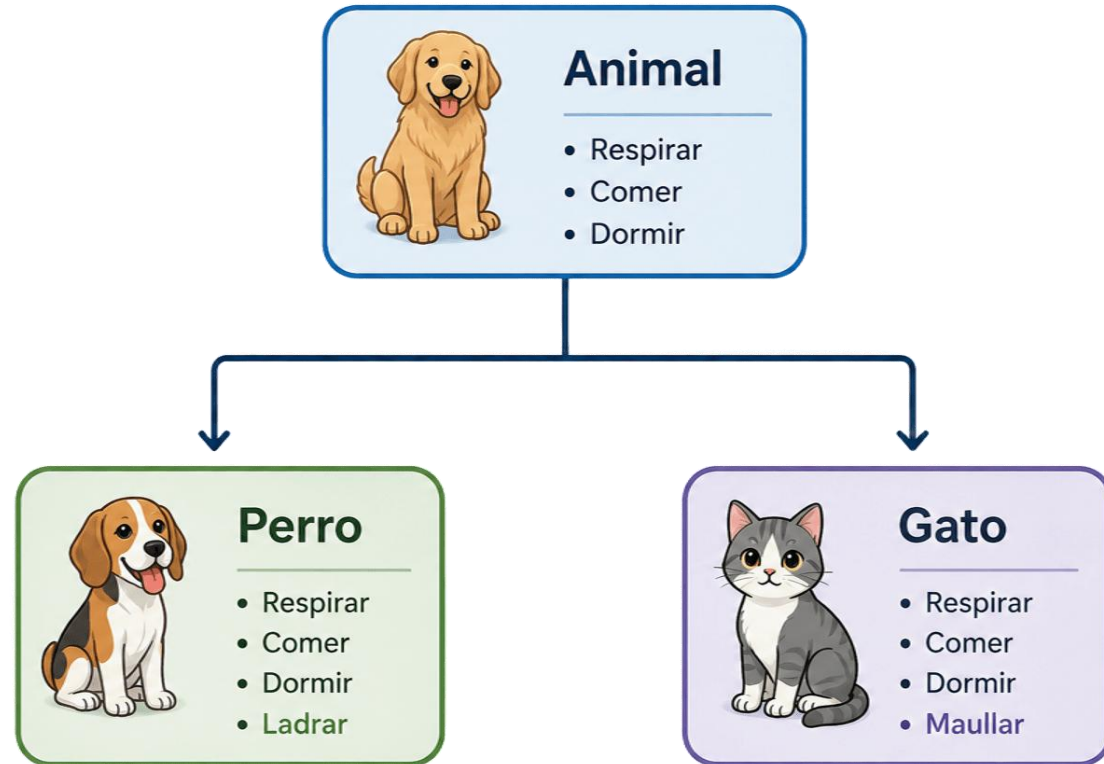


Características heredadas:

- Nombre
- Edad
- Moverse

Idea clave:

Las clases hijas heredan características de una clase más general y pueden agregar sus propias funciones.



La herencia permite crear nuevas clases a partir de otras existentes, reutilizando atributos y comportamientos comunes.

Ejemplo

Método:

`hacer_sonido()`

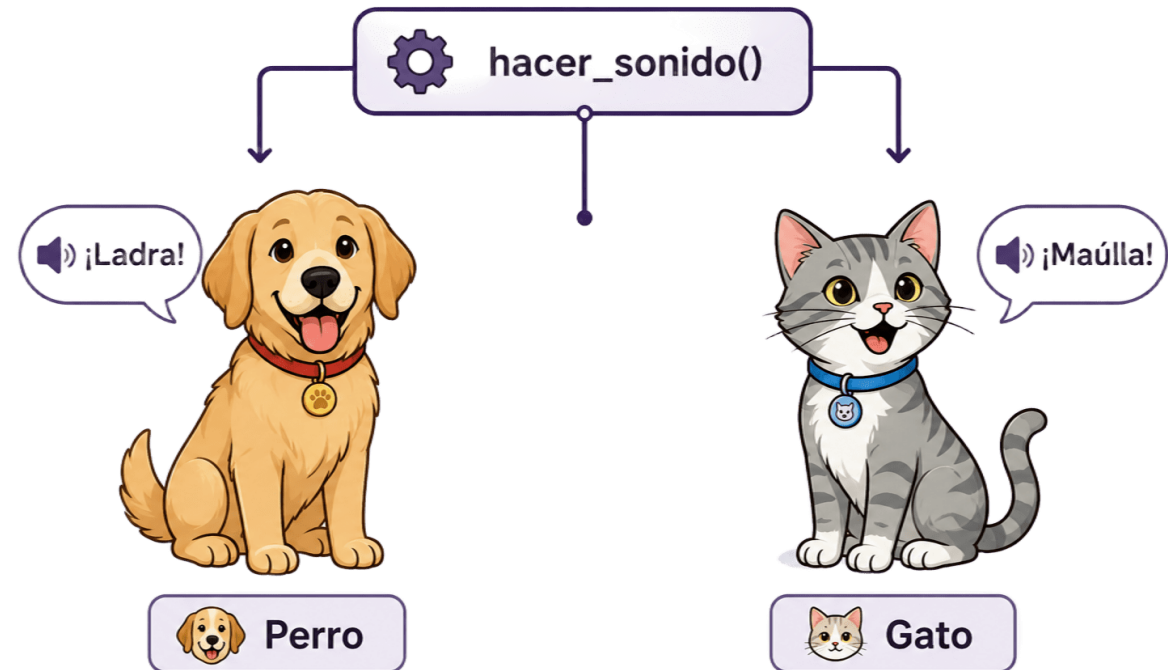
Resultado:

🐶 Perro → Ladra

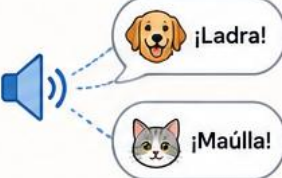
🐱 Gato → Maúlla

Idea clave:

Una misma acción puede producir resultados diferentes según el objeto que la ejecute.



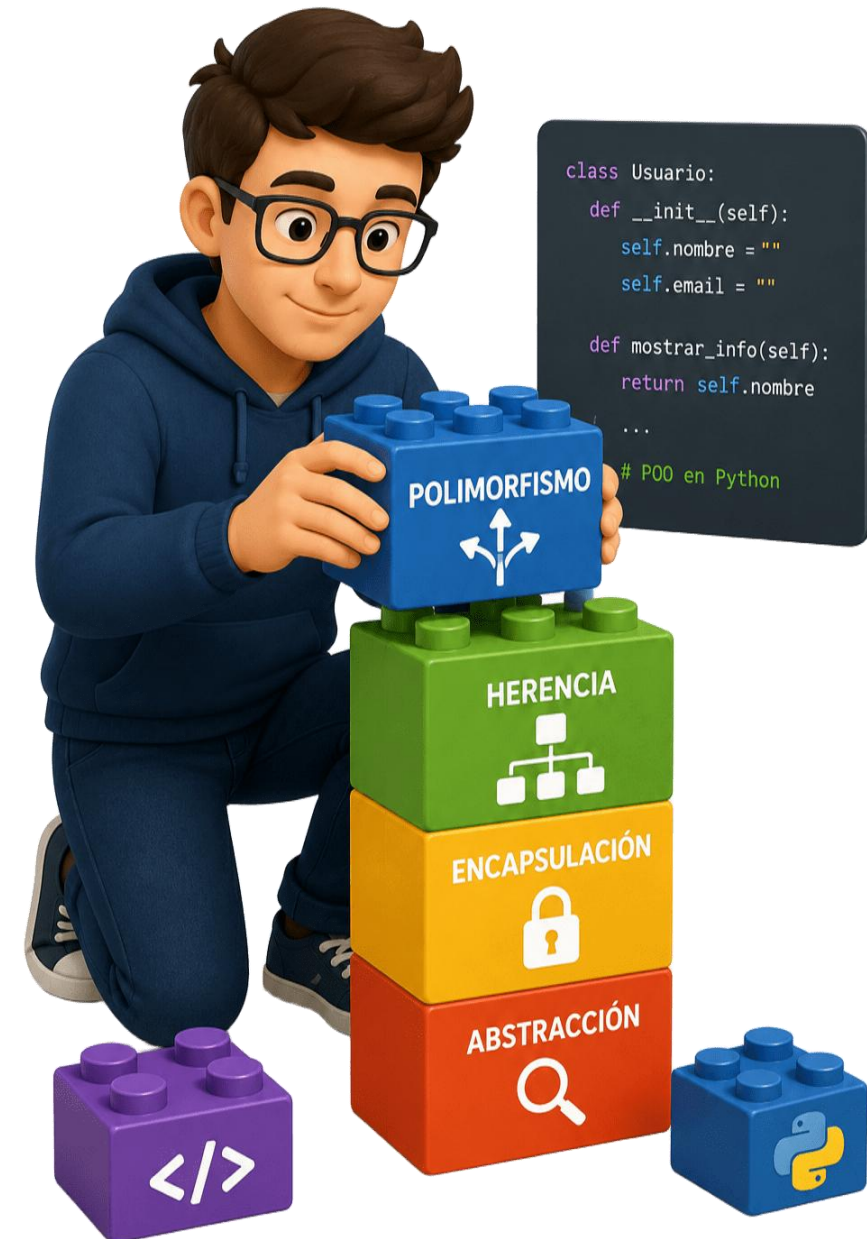
¿DÓNDE SE APLICAN ESTAS TÉCNICAS EN PYTHON?

TÉCNICA	APLICACIÓN	¿CÓMO SE USA EN PYTHON?
 Abstracción	Clases 	<pre>class Automovil: def __init__(self, marca, modelo, velocidad): self.marca = marca self.modelo = modelo self.velocidad = velocidad</pre> Se crea una clase que representa lo esencial de un objeto, ignorando los detalles internos. 
 Encapsulación	Atributos y métodos 	<pre>class CuentaBancaria: def __init__(self, saldo): self.__saldo = saldo # Atributo privado def depositar(self, monto): self.__saldo += monto def obtener_saldo(self): return self.__saldo</pre> Se protegen los datos internos controlando el acceso a ellos mediante métodos. 
 Herencia	Clases derivadas 	<pre>class Animal: def __init__(self, nombre): self.nombre = nombre class Perro(Animal): def __init__(self, nombre, raza): super().__init__(nombre) self.raza = raza</pre> Se crean nuevas clases que heredan atributos y métodos de otra clase existente. 
 Polimorfismo	Métodos redefinidos $fx()$	<pre>class Animal: def hacer_sonido(self): pass class Perro(Animal): def hacer_sonido(self): return "Ladra" class Gato(Animal): def hacer_sonido(self): return "Maúlla"</pre> Distintas clases pueden implementar el mismo método de diferentes formas. 

REFLEXIÓN

Las técnicas de la Programación Orientada a Objetos permiten modelar problemas reales de forma organizada y eficiente.

Su correcta aplicación facilita el desarrollo de software moderno, reutilizable y fácil de mantener.





Conclusiones

- La Programación Orientada a Objetos se basa en cuatro técnicas fundamentales: abstracción, encapsulación, herencia y polimorfismo
- Cada técnica ayuda a organizar y mejorar el diseño del software.
- Python permite aplicar estas técnicas de forma sencilla y práctica.
- Estos conceptos serán la base para desarrollar aplicaciones orientadas a objetos durante el curso.

Hernández Bejarano, M., & Baquero Rey, L. E. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.

Hernández Bejarano, M., & Baquero Rey, L. E. (2025). Python con orientación a objetos y al análisis de datos. Ediciones de la U.

Bibliografía

Boucheny, V. (2025). Aprender programación orientada a objetos con Python (3.^a ed.).

Ediciones ENI. Piñeiro Gómez, J. M. (2022). Principios de la programación orientada a objetos. Paraninfo.



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

JUNTOS TRANSFORMAMOS EL MUNDO DESDE LA AMAZONÍA



**CARRERAS
EN LÍNEA**

***Dr. David Sancho, PhD.
RECTOR UEA***